EXAMINATION RESULT AND GRACE-MARK VALIDATION

Python Project Report

Project Overview

This project processes marks across multiple subjects, applies subject-wise pass criteria and a grace-mark policy, validates mark limits, and generates each student's final classification.

Example policy used

Pass mark is 35 out of 100 in each subject. A student may receive up to 5 grace marks in total, only to bring a subject that is within 5 marks of passing up to 35. Marks above 100 are invalid and are not accepted as final marks. These are sample rules because no exact policy values were supplied.

1. Problem Statement

Develop a Python program to process student marks in multiple subjects, check subject-wise pass criteria, apply the specified grace-mark rule, validate maximum marks, and classify the final result.

2. Objectives

• Read marks for each subject for multiple students.

• Check that each mark is between 0 and 100.

• Apply grace marks only when allowed by the policy.

• Ensure final marks never exceed the maximum marks.

• Calculate total final marks and classify each student as PASS, FAIL, or INVALID RESULT.

3. Assumptions and Rules

• Each subject has a maximum of 100 marks.

• A subject pass mark is 35.

• The grace-mark limit is 5 marks per student across all subjects.

• Grace marks are used only if they can bring a subject to the pass mark within the remaining grace limit.

• A mark below 0 or above 100 is invalid.

• PASS means every subject is passed; FAIL means at least one valid subject is failed; INVALID RESULT means at least one subject has invalid marks.

4. Algorithm

1. Start.

2. Read each student's marks in all subjects.

3. Validate every mark against the range 0–100.

4. If a mark is invalid, classify that subject as INVALID MARKS and do not add it to the total.

5. For valid marks, check whether the student passed the subject.

6. If marks are just below 35, apply grace marks if the total grace limit allows it.

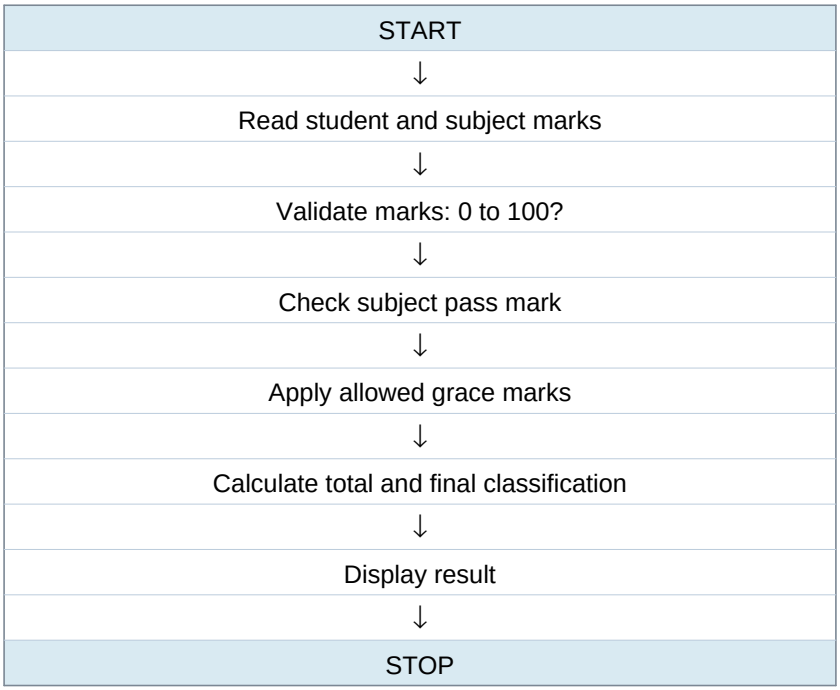
7. Calculate final total using valid final marks.

8. Classify the student as PASS, FAIL, or INVALID RESULT.

9. Display subject-wise marks, grace used, total, and classification.

10. Stop.

5. Flowchart



6. Pseudocode

```
BEGIN
    INPUT subject marks for each student
    grace_used = 0
    FOR each subject:
        IF marks < 0 OR marks > 100:
            status = INVALID MARKS
            do not add marks to total
        ELSE:
            IF marks >= 35:
                status = PASS
            ELSE IF marks is within grace policy
                 AND grace limit is available:
                add required grace marks, without exceeding 100
                status = PASS (GRACE)
            ELSE:
                status = FAIL
        calculate total from valid final marks
    IF any invalid marks: classification = INVALID RESULT
    ELSE IF all subjects passed: classification = PASS
    ELSE: classification = FAIL
    DISPLAY subject results and classification
END
```

7. Python Source Code

Save the program as **examination_result_grace_mark_validation.py**.

```
# Examination Result and Grace-Mark Validation

def process_student(name, subjects, grace_limit=5, pass_mark=35, max_mark=100):
    results = []
    grace_used = 0
    total = 0
    all_pass = True

    for subject, marks in subjects.items():
        if marks < 0 or marks > max_mark:
            results.append((subject, marks, "-", "INVALID MARKS"))
            all_pass = False
            continue

        final_marks = marks
        status = "PASS" if marks >= pass_mark else "FAIL"

        # Apply grace marks only to a subject just below the pass mark.
        if status == "FAIL" and marks >= pass_mark - grace_limit:
            needed = pass_mark - marks
            if grace_used + needed <= grace_limit:
                final_marks = min(marks + needed, max_mark)
                grace_used += needed
                status = "PASS (GRACE)"

        if status.startswith("FAIL"):
            all_pass = False

        results.append((subject, marks, final_marks, status))
        if isinstance(final_marks, (int, float)):
            total += final_marks

    if any(row[3] == "INVALID MARKS" for row in results):
        classification = "INVALID RESULT"
    elif all_pass:
        classification = "PASS"
    else:
        classification = "FAIL"

    return results, total, grace_used, classification


def main():
```

7. Python Source Code (continued)

```
    students = [
        ("Anu", {"English": 72, "Maths": 33, "Science": 68}),
        ("Ravi", {"English": 80, "Maths": 55, "Science": 29}),
        ("Sara", {"English": 91, "Maths": 78, "Science": 85}),
        ("Kiran", {"English": 105, "Maths": 60, "Science": 70})
    ]

    print("=" * 82)
    print("            EXAMINATION RESULT AND GRACE-MARK VALIDATION")
    print("=" * 82)
    print("Policy: Pass mark = 35/100 | Total grace limit = 5 marks per student")
    print("-" * 82)

    for name, subjects in students:
        results, total, grace_used, classification = process_student(name, subjects)
        print(f"\nStudent: {name}")
        print(f"{'Subject':<12}{'Original':>10}{'Final':>10}   Status")
        for subject, original, final, status in results:
            print(f"{subject:<12}{str(original):>10}{str(final):>10}   {status}")
        print(f"Total Final Marks: {total}")
        print(f"Grace Marks Used: {grace_used}")
        print(f"Final Classification: {classification}")

    print("\n" + "=" * 82)
    print("Result processing completed successfully.")

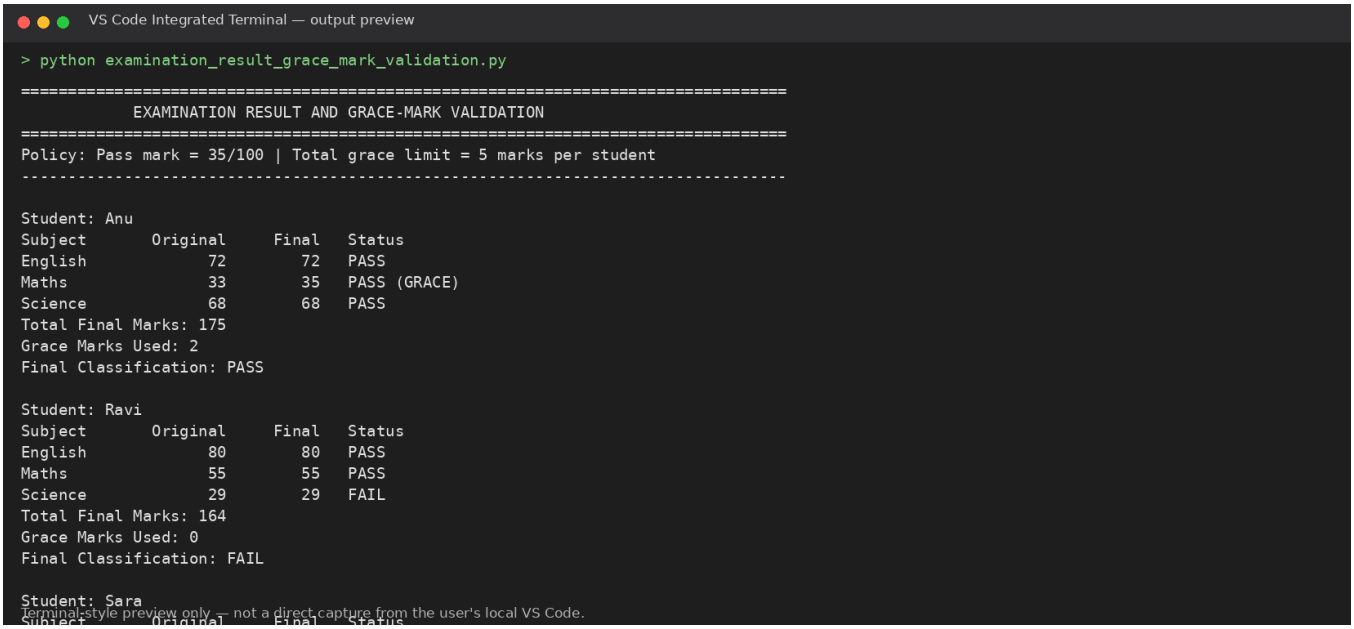

if __name__ == "__main__":
    main()
```

8. Test Cases

Test case	Input / condition	Expected result
Normal pass	All subjects ≥ 35	PASS
Grace marks	33 marks; pass mark 35; grace available	Final mark 35; PASS (GRACE)
Failed subject	29 marks; grace policy cannot cover it	FAIL
Maximum mark violation	105 out of 100	INVALID MARKS; not counted as final marks
All subjects passed	All marks at or above pass mark	PASS

9. Output Screenshot

This is a terminal-style output preview, not a direct screenshot from the user's local VS Code.



For a genuine VS Code screenshot, open the Python file, select Terminal → New Terminal, run **python examination_result_grace_mark_validation.py**, and take a screenshot of the editor and terminal.

10. Conclusion

The program checks subject marks, applies a limited grace-mark policy, rejects marks outside the allowed range, calculates final totals, and classifies results. The sample pass and grace rules can be changed to match the exact policy supplied by the institution.